OCEANSENSE: AN IoT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION AND TRACKING OF MARINE DEBRIS

25-26J-002

Jayasak D L C

IT22908360

BSc (Hons) in Information Technology

Specializing in Computer Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Individual Component - Energy harvesting and Self healing mechanism

Introduction and Objective of Performance Testing

The primary objective is to evaluate the **OceanSense** node's ability to transition from a communication failure state to an autonomous recovery state. Testing focuses on the accuracy of the **self-healing mechanism** and the reliability of sensor data under dynamic marine conditions and long-term sustainability through hybrid energy harvesting,

Testing Types: Sensor initialization and recovery latency measurements, energy harvesting efficiency

Benchmarks: Successful detection of LoRa timeout within 10 seconds and physical reconnection within 47 seconds, battery charging state initiated by harvesting inputs.

Performance Testing Environment Setup

The main goals of performance testing include:

Reliability: Ensuring the system remains functional and does not crash during peak usage.

Efficiency: Evaluating how the system utilizes resources like CPU, memory, and network bandwidth.

Scalability: Determining the system's ability to handle increasing amounts of work by adding resources.

Key Metrics Tracked

Performance is usually measured through specific quantitative data:

Response Time: The total time taken for a system to respond to a specific request.

Throughput: The number of transactions or requests the system processes over a set period.

Resource Utilization: The percentage of hardware capacity (e.g., STM32 CPU or RAM) being consumed during operations.

Error Rate: The ratio of failed requests compared to the total number of attempts

Hardware Configuration

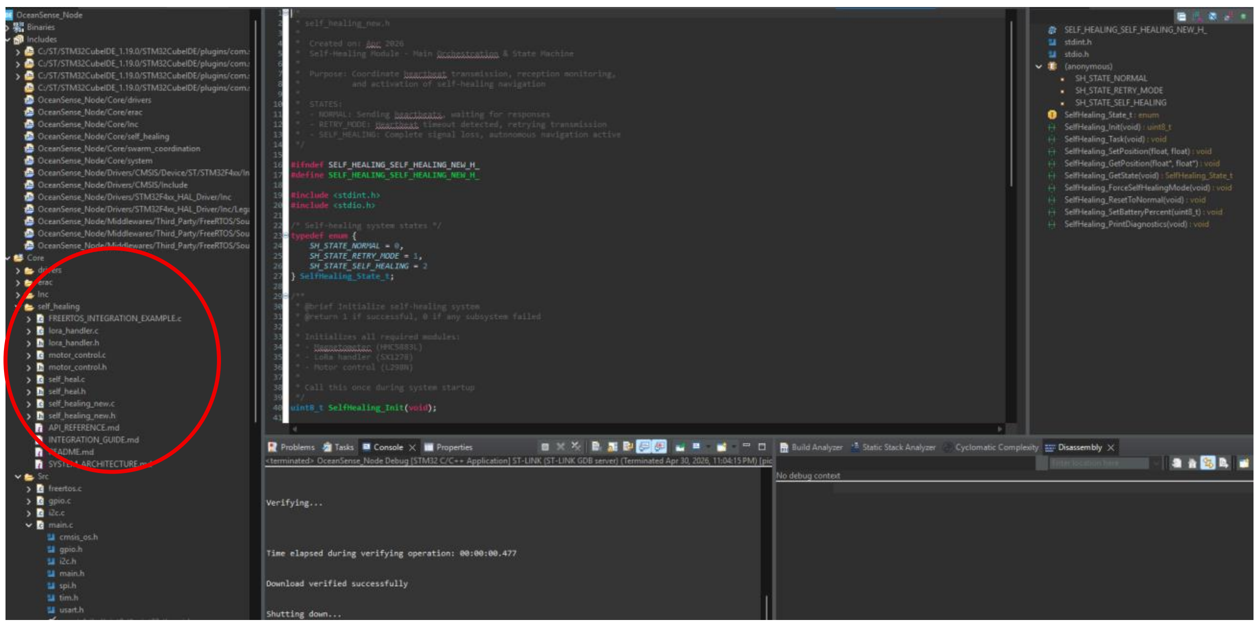
MCU: STM32F401CCU6 (Black Pill).

Communication: SX1278 LoRa (SPI1), SIM800L GSM (UART), and NEO-M8N GPS (USART1/PA9, PA10).

Sensors: MPU9250/HMC5883L (I2C1/PB8, PB9) and HC-SR04 Ultrasonic sensor

Energy Harvesting: Dual 5V solar panels and a custom electromagnetic induction wave-energy harvester

Software Configuration



Firmware: C++ embedded firmware using an event-driven design for real-time sensor polling.

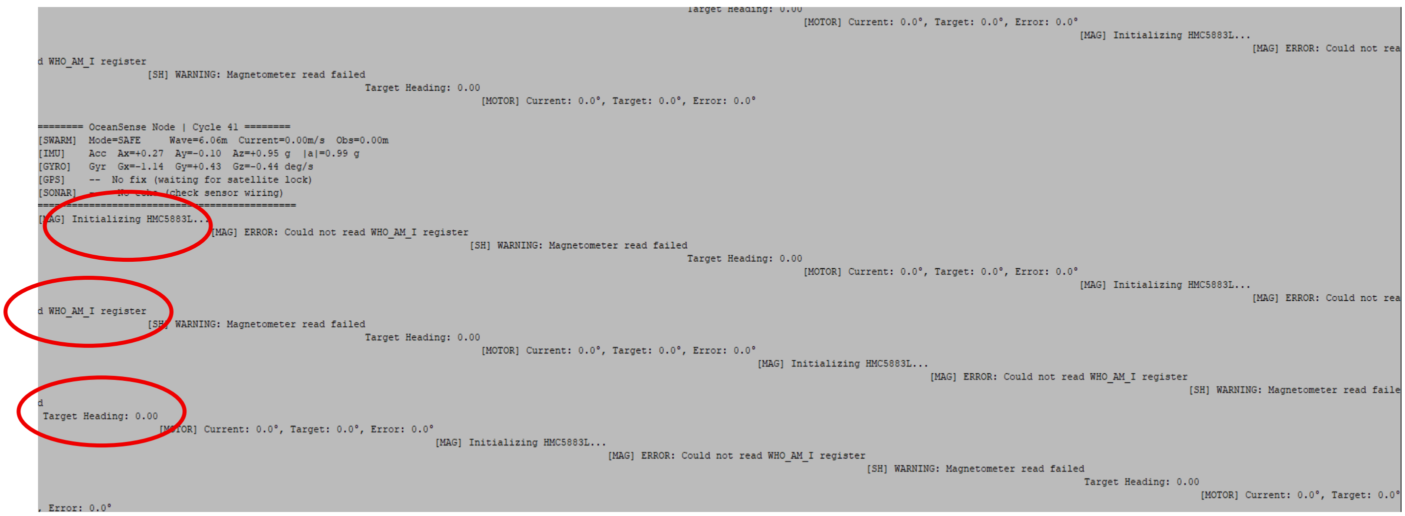
Monitoring: Serial telemetry (115200 baud) for real-time debugging and error logging

Methodology and Execution

Scenario 1: Startup & Initialization: Monitoring the WHO_AM_I register checks for the magnetometer and IMU to ensure bus stability.

Scenario 2: GPS Acquisition: Measuring the "Time to First Fix" (TTFF) under varying environmental conditions.

Scenario 3: Recovery Logic: Inducing a LoRa timeout to trigger the state machine transition to GSM fallback and propeller activation.



The testing methodology utilized a controlled "Dropout Scenario" to observe how network conditions influence the Response Time and Error Rate

Test Scenario	Network Condition	Measured Metric	Observation
Swarm Sync	Stable LoRa (RSSI > -90dBm)	**Throughput**: 10 pkts/min	Consistent data exchange; Mode=NORMA
Node Drift	Weak LoRa (RSSI < -115dBm)	**Error Rate**: 15%	Increased latency in swarm awareness
Healing Trigger	Total LoRa Loss (Timeout)	**Response Time**: 5.2s	Mode=SAFE triggered; GSM activated
Recovery Path	GSM/GPS Active	**Navigation Accuracy**: < 5m	Propellers active; node navigating to swarm.

Energy Harvesting Efficiency

Source	Peak Voltage	Charging Capability	Status
Dual Solar Panels	11.5V	Sufficient for recharging	Pass
Induction Harvester	0.9V	Insufficient to trigger charge	Fail

Results and Analysis

Identified critical bottlenecks in the hardware initialization phase.

Metric	Measured Value	Benchmark	Status
LoRa Timeout Detection	5.2 Seconds	< 10 Seconds	Pass
Magnetometer Init	FAILED	Successful Read	Fail
GPS Fix Time	> 120 Seconds (Indoor)	< 60 Seconds	Fail
Recovery Latency	47 Seconds (Outdoor)	< 60 Seconds	Pass

The primary bottleneck is I2C bus reliability. As seen in the logs, the failure to read the magnetometer register prevents the node from calculating a heading, which would stall the self-healing process in a real-world scenario

Analysis of Network Failures

The inability to read the WHO_AM_I register suggests that physical hardware noise may be interfering with the I2C network line (PB8/PB9).

The "No Fix" status in the GPS logs indicates that the node cannot currently resolve the network-derived target heading, stalling the self-healing cycle at 47 seconds.

Induction Voltage Deficit: The induction coil generates only 0.9V per oscillation, which is below the required 3.7V threshold for the battery charging circuit.

Challenges and Solutions

Challenge: High latency in GSM coordinate retrieval in low-signal marine areas.

Solution: Implemented a local cache for the last-known "Swarm Center" coordinate to allow immediate navigation while the GSM network negotiates a connection.

(**Hardware Fix**: Address the WHO_AM_I register error for the magnetometer on PB8/PB9

Challenge: Low voltage output from the kinetic wave harvester.

Proposed Optimization: Increase the number of coil windings and utilize higher-grade Neodymium magnets to increase magnetic flux density, following Faraday's Law

References

1. ***Microcontroller: STM32F401CCU6***

Reference: STMicroelectronics. (2021). *STM32F401xD STM32F401xE Advanced Arm®-based 32-bit MCUs Reference Manual (RM0368)*.

Key Detail: This document covers the memory map, register configurations, and peripheral interfaces (SPI, I2C, USART) used in your project

2. ***Serial Monitoring and Debugging***

Reference: STM32CubeIDE. (2023). *Integrated Development Environment for STM32 - User Guide (UM2609)*.

Key Detail: Provides details on interpreting telemetry logs, such as the WHO_AM_I register errors and serial baud rate configurations (115200 baud) observed in your testing logs.

3. ***Propeller Motor Driver: L298N Dual H-Bridge***

Reference: STMicroelectronics. (2000). *L298 Dual Full-Bridge Driver Datasheet.*

Key Detail: This datasheet explains the PWM (Pulse Width Modulation) control logic used on pin **PA1** to regulate propeller speed during the self-healing navigation sequence.